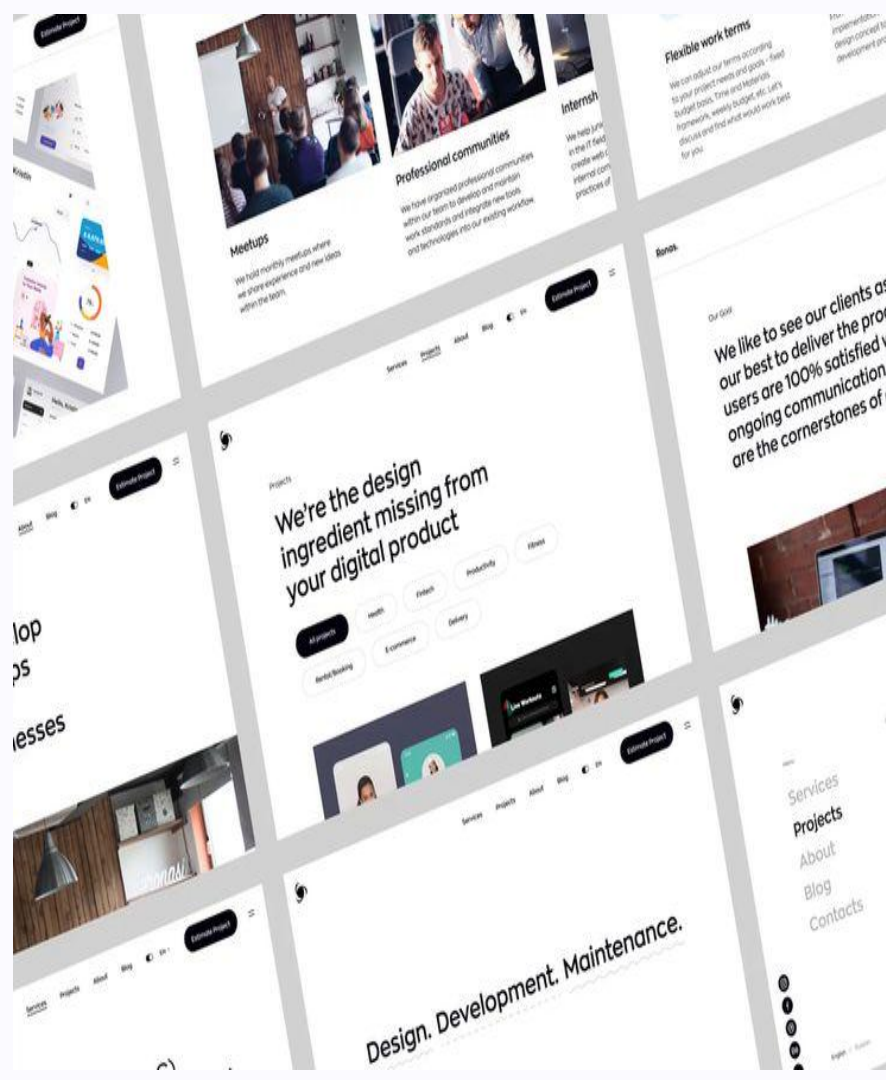


PROJECT 16

The Tab Hoarder

A browser tab simulator built entirely within a single webpage — exploring state management, UI constraints, and data persistence.



What is The Tab Hoarder?

The Tab Hoarder is a frontend application that simulates browser tab management — **fake tabs** — directly inside a single webpage. Rather than relying on the browser's native tab system, it recreates the full tab experience: opening, switching, closing, and restoring — all governed by custom rules.

State Management

Tracks active and archived tab content in real time.

UI Constraints

Enforces a strict 4-tab limit with automated eviction.

Data Persistence

Survives page refreshes using localStorage.

Core Rules & Automation

The Tab Hoarder operates under three strict rules that define its behavior and create a reactive, self-managing UI.



Open 5th Tab

Auto-close Oldest

Move to Graveyard

Every user action triggers a predictable chain of events — keeping the interface clean and the data model consistent.

Where Closed Tabs Go

When a tab is closed — whether manually by the user or automatically by the 4-tab limit — its content is **never lost**. It is immediately moved to a dedicated [Graveyard section](#) below the active tab area.

This creates a visible history of all evicted content, giving users the ability to review, recover, or reference anything that has been cycled out.

Restoring from the Graveyard

The Duplication Rule

Restoring any tab from the Graveyard doesn't just bring it back — it **duplicates its content twice**, creating two new active tabs with identical content.

This intentional quirk adds a layer of complexity to state management and reinforces the constrained, almost game-like nature of the application.

Restored tabs re-enter the ring buffer and are subject to the same 4-tab eviction rules.

CSS Pseudo-Classes & Transitions

The visual layer relies heavily on CSS pseudo-classes to communicate tab state without JavaScript intervention for rendering.



Active / Inactive States

`:active` and custom class toggling distinguish the focused tab from background tabs visually.



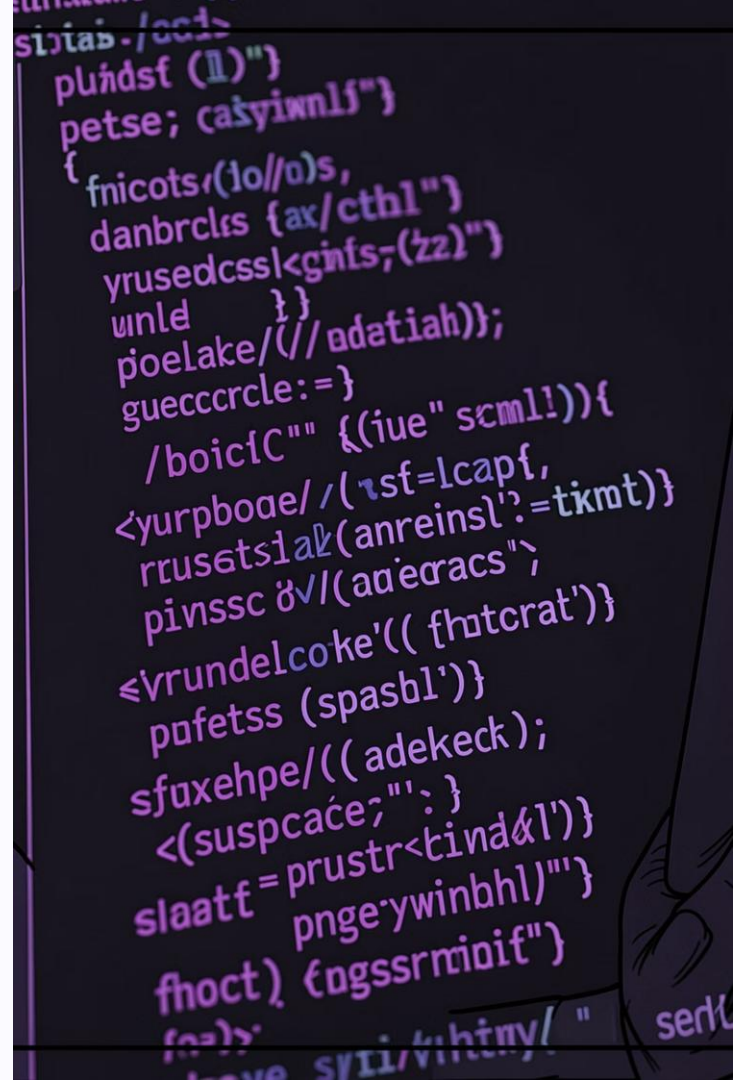
Hover Effects

Smooth `:hover` transitions on tab headers provide clear affordance before interaction.



Enter / Exit Animations

CSS transitions animate tabs sliding in and fading out, reinforcing the sense of a real tab system.



Ring Buffer in JavaScript

The 4-tab limit is enforced using a [circular array \(ring buffer\)](#) — a fixed-size data structure that overwrites the oldest entry when full.

When the user opens a 5th tab, the buffer automatically cycles: the oldest tab's data is popped, its content sent to the Graveyard, and the new tab appended at the end.

- ❑ Ring buffers provide $O(1)$ insertion and eviction — ideal for a strict, rolling window of active items.

localStorage: Progress That Survives Refresh

Both the active tab buffer and the Graveyard list are serialized and saved to localStorage on every change. On page load, the application hydrates its state from storage — meaning no tab content or graveyard history is ever lost unintentionally.

Active Tabs Saved

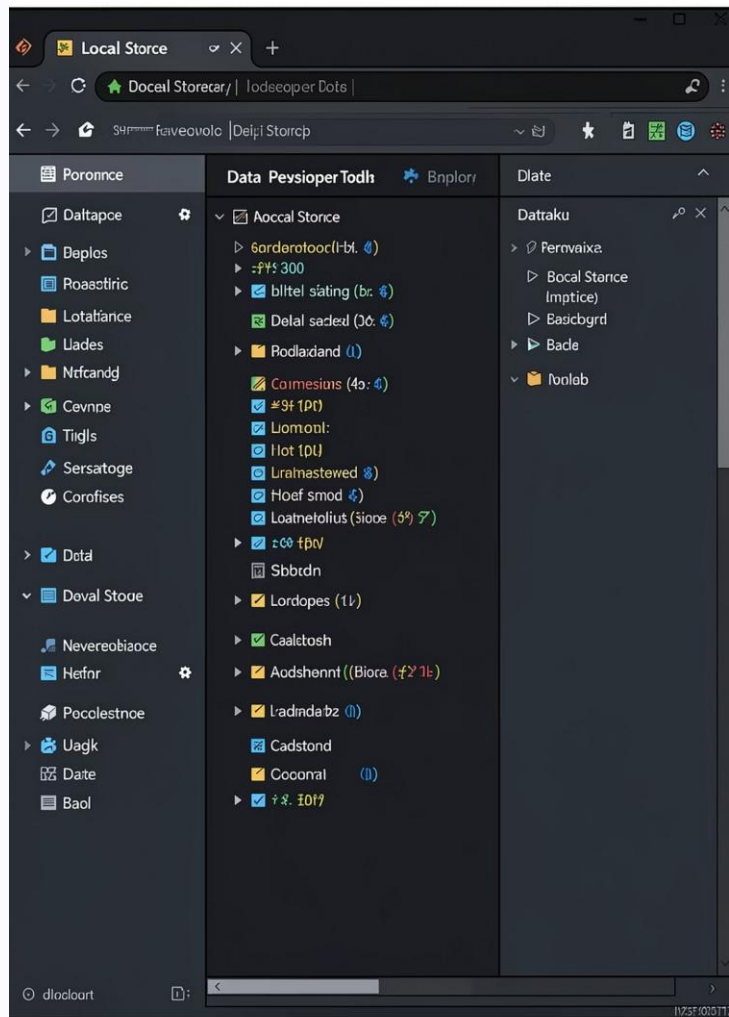
The full ring buffer array is stringified and stored on each tab change.

Graveyard Persisted

Closed tab content is stored as a separate array, maintaining full history.

Hydration on Load

On mount, the app reads from localStorage and reconstructs the exact previous state.



Technology & Architecture at a Glance

The Tab Hoarder is a pure frontend exercise — no backend, no framework dependencies. Every constraint is intentional, designed to deepen understanding of core web APIs and reactive UI patterns.



01

HTML Structure

Semantic tab container, active panel, and graveyard section.

02

CSS Layer

Pseudo-classes, transitions, and state-driven styling.

03

JavaScript Logic

Ring buffer, event handling, and localStorage sync.



Key Takeaways & Next Steps

The Tab Hoarder demonstrates that meaningful complexity can emerge from simple constraints. Managing state, enforcing rules, and persisting data — all without a framework — sharpens core frontend skills.

Constrained UI Design

Limits force creative, predictable patterns.

State Without a Framework

Vanilla JS ring buffer handles all tab lifecycle logic.

Persistence Matters

localStorage transforms a demo into a usable tool.